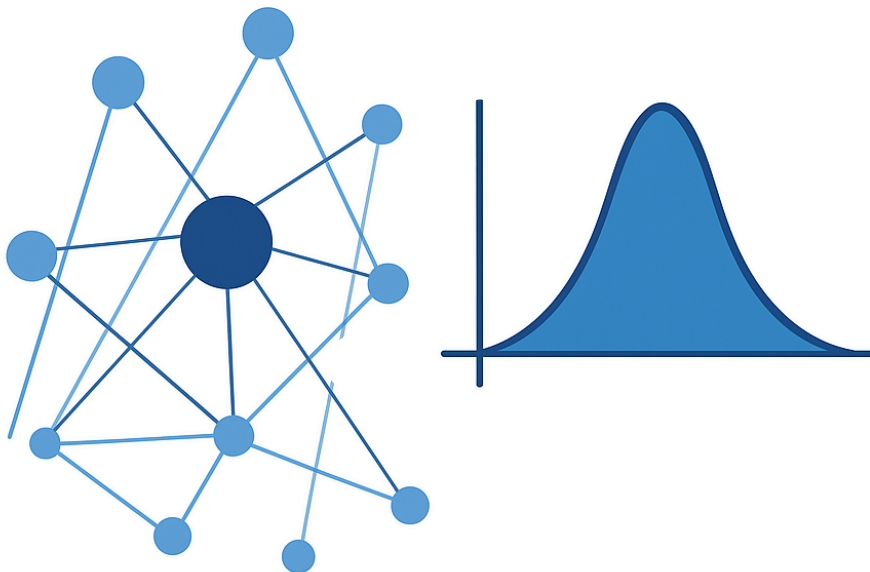


Probabilistic Thinking in Quant Trading: Essential Tools and Models

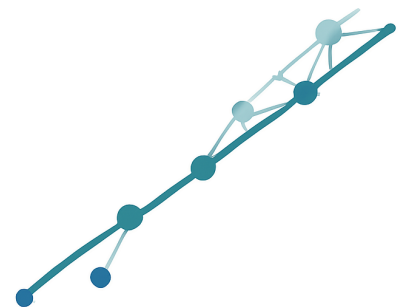
Exploring how to quantitatively optimise position sizing, regime resilience, signal confidence, and more



Author: William Nelson
Created: 10 July 2025
Last Updated: 14 July 2025

Table of Contents

Title Page.....	1
Table of Contents.....	2
Introduction.....	3
Kelly Criterion.....	4
Bayesian Inference.....	5
Decision Trees.....	6
Markov Models.....	7
Signal Stacking and Distributional Thinking.....	8
Conclusion.....	9



Introduction

In Quant trading, uncertainty is not a nuisance - it is the medium in which all decisions are made. Unlike deterministic systems where cause and effect are directly observable, we know markets are not like that, due to noise, lack of information, and constant shifting dynamics. In this environment, thinking probabilistically is essential.

Probabilistic thinking refers to a framework that allows traders to navigate uncertainty, update beliefs in light of new info, and optimise decisions based on expected outcomes.

This research piece explores several of these foundational tools - from the Kelly Criterion for optimal bet sizing, to Bayesian Inference for belief updating, to regime-switching models that reflect the non-stationary nature of markets. The goal is to articulate a mental toolkit: a set of probabilistic ideas that every serious quant should understand and apply.

The Kelly Criterion: Growth-Optimal position sizes

The Kelly Criterion is a foundational concept for probabilistic finance. It offers a mathematical framework for optimal capital allocation under uncertainty. It is a cornerstone in areas ranging from bet sizing in gambling to portfolio construction.

At its heart, Kelly is about maximising long term wealth growth by betting a fraction of one's capital in proportion to their edge and the odds. Its a probabilistic tool (not just a formula) that formalises how to act under uncertainty when outcomes are repeatable and capital is limited.

Here's the idea:

Imagine you're presented with a favourable bet multiple times - say, betting one a biased coin that lands heads 60% of the time. How much should you wager each time to maximise your capital over the long run?

- Too little - underutilise edge
- Too much - vol might run you over

The Kelly solution is the mathematical sweet spot: the bet size that maximises the expected logarithmic growth of your capital.

The formula is:

Fraction of capital to wager = expected excess return / variance of return

Key characteristics

The Kelly Criterion assumes we care about compounding. It maximises the geometric mean of wealth over time - not EVs of single outcomes.

Its logarithmic in nature, meaning it is extremely sensitive to ruin. Kelly avoids strategies that lead to losing everything, even if the average return looks attractive.

Kelly is often used in fractional form in portfolios to reduce drawdowns and smooth vol due to estimation error.

Example of Kelly Criterion built into a system:

Imagine you've developed a mean-reversion signal on SPY, triggered when price falls below its 10 day sma by 2SD.

You backtest:

- 55% win rate, avg gain on losing trades +1.5%, average loss on losing trades -1.0%

If we plug these results into Kelly we get 0.25. this means our optimal bet sizing is 25% of total capital per trade.

Bayesian Inference: Updating Beliefs

In systematic trading, few beliefs remain fixed. Markets evolve and relative values dislocate. To operate effectively in this uncertain and dynamic environment, traders must continuously reassess what they believe - and how strongly they believe it. This is where Bayesian Inference plays a role.

At its core, Bayesian inference is a mathematical framework for updating beliefs based on new evidence. Rather than treating parameters or outcomes with fixed truths, it treats them as probability distributions that evolve with information. This aligns naturally with the needs of systematic trading, where data is noisy and every conclusion must come with a measure of uncertainty.

$$P(\theta|\text{data}) = P(\text{data}) \cdot P(\theta) / P(\text{data}|\theta)$$

Where θ is the parameter or hypothesis of interest

What Bayesian Inference does, is converts backtesting into an evolving confidence score.

So where do I actually use this in Quant?

Especially useful in low sample strategies, you can use a Bayesian posterior distribution to reflect confidence.

You can also use it to continuously update your belief about a system's edge based on new PnL data.

Example of Bayesian Inference built into a system:

Imagine you have a moment system, you've only observed a small number of live trades or small number of backtested trades. Rather than assuming the signal is solid (poor assumption), you use Bayesian Updating to gradually build confidence.

First model the mean return of the signal and produce a normal distribution. You then use the backtested data in the formula to solve for posterior belief. You constantly update this as more trades are executed to see the change in posterior belief, this will see SD decreasing which gives your returns on the system more statistical confidence, which means you can pair this with position scaling (possibly combined with Kelly).

Bayesian updating tells you how confident you should be in your system, its important to scaling.

Probability Trees: The Strategy Chooser

Uncertainty in trading isn't just about not knowing what will happen - its about facing multiple possible futures, each with its own path, likelihood, and consequence. Probability trees are a structures way to represent these paths, allowing quants to map out decisions and outcomes over time and calculate the EVs of different strategies under uncertainty.

A decision tree (which is a type of ML) is a branching diagram that shows all the possible outcomes of a sequence of events, along with the probability and payoff of each outcome. Each node represents a decision point or stochastic event. The branches represent possible outcomes from these nodes and their probabilities. By traversing the tree, you can compute:

- EVs (most important output)
- Conditional paths
- Worst case and best case scenarios
- Path-dependant payoffs

So where do I actually use this in Quant?

Since even a single tree can be tested with multiple conditional signals, using the EVs, you can rank all strategies used on the decision trees by Sharpe, then select the top Sharpe strategies.

In reality though what a quant trader would do is feed a trees-based system (think random forests, LightGBM) data, then give it a condition like "go long if 1-day EV exceeds 0.5%", then they ask the system to tell them the exact feature conditions that led to this return being met correctly. They can then run this on a bunch of EV parameters and filter by Sharpe and then see what features/conditions are Sharpe optimal.

Outside of strategy selection and system testing, decision trees are great for option pricing and system stress testing (using multi-asset trees to simulate adverse macro paths), and more.

Decision trees require lots of computer power to run, but they are very powerful.

Example of Decision Trees built into a system:

Lets use XGBoost (uses decision trees) as an example, you feed a bunch of data + features (with care and insight) into XGBoost to train it, the decision trees will learn what combination of signals from the features will return the best EVs, and it will use this strategy.

Markov models: The regime detectors and adapters

Markets are not stationary. What works in a bull market will fail catastrophically in a bear market. A strat optimised for low vol may struggle in high vol regimes. Recognising that market dynamics evolve over time is very important to model consistency and regime switching models aim to capture this.

One of the most common approaches to regime modelling is Markov models - especially the Hidden Markov Models (HMMs) - which assume that the systems transitions probabilistically between regimes, with data like returns and vol being influenced by the current state.

Markov process assumes there is a regime-transition probability matrix, which determines the likely next regime.

The HMMs tell use 'hidden' regimes, that are harder to spot (especially in noisy market data), they also forecast likely regime transitions and adapt strategy weights accordingly.

So where do I actually use this in Quant?

Lets say you're running two systems in your portfolio, a momentum one and a mean-reversion system. Now depending on regime, one can perform better than the other.

By applying a HMM, you estimate a 70% chance of the current regime being trending. However the forecast for next day says theres a 60% chance trending continues, 40% chance it flips. So based off this information you adjust for slightly more capital allocation in mean-reversion system.

Signal Stacking in Practice

Quant desks rarely rely on a single predictor. Instead they maintain libraries of dozens of weak signals that fire in different regimes, asset classes, and time-scales. The playbook below shows how pros merge those fragments of information into a composite market view.

- Standardise each signal (produces a 'score')
- Apply ridge across the signals to shrink weights
- Might add an ML overlay
- Map the composite score to probability of positive returns (combine with Kelly)

Why Quants think in Distributions

Discretionary traders will often ask “wheres the market gonna close”, quants are asking “what is the distribution of possible closes and how should I position across it?”. This kind of stochastic framework enables proper risk-adjusted sizing (Kelly), supports scenario thinking (stochastic, think decision trees), and facilitates Bayesian updating to see whether your model can handle the market close distribution.

So stop asking “will EUR/USD go up?” and start asking “how much capital should I allocate too each percentile of EUR/USD possible moves?”.

Modelling uncertainty is what its all about.

Conclusion

- Kelly sizing - shows how to allocate capital proportionally to edge and risk
- Bayesian Updating - offers a way to update confidence in your signals
- Decision trees - help choose conditions from features
- Markov models - allow your system to adapt to regime transitions
- Signal stacking - transforms scattered insights into robust composite signals